Java 虚拟机面试题

大家好，我是小林。

JVM 这块在面试里属于"加分项"，但如果你答不上来，也会很明显拉低印象分。尤其是做 Java 后端，被问到内存溢出怎么排查、GC 为什么频繁、类是怎么加载进来的，这些问题你总归要面对。很多人用 Java 写了好几年代码，但对 JVM 的了解只停留在"知道有个垃圾回收"，一旦被追问细节就说不清楚了。

这篇文章整理了 JVM 面试中最常被问到的知识点，涵盖内存模型、类加载机制、垃圾回收算法和垃圾回收器这几块核心内容。内容偏向理解原理，帮你搞清楚 JVM 在背后到底做了什么。

几个在面试里出现频率最高的方向，建议重点花时间：

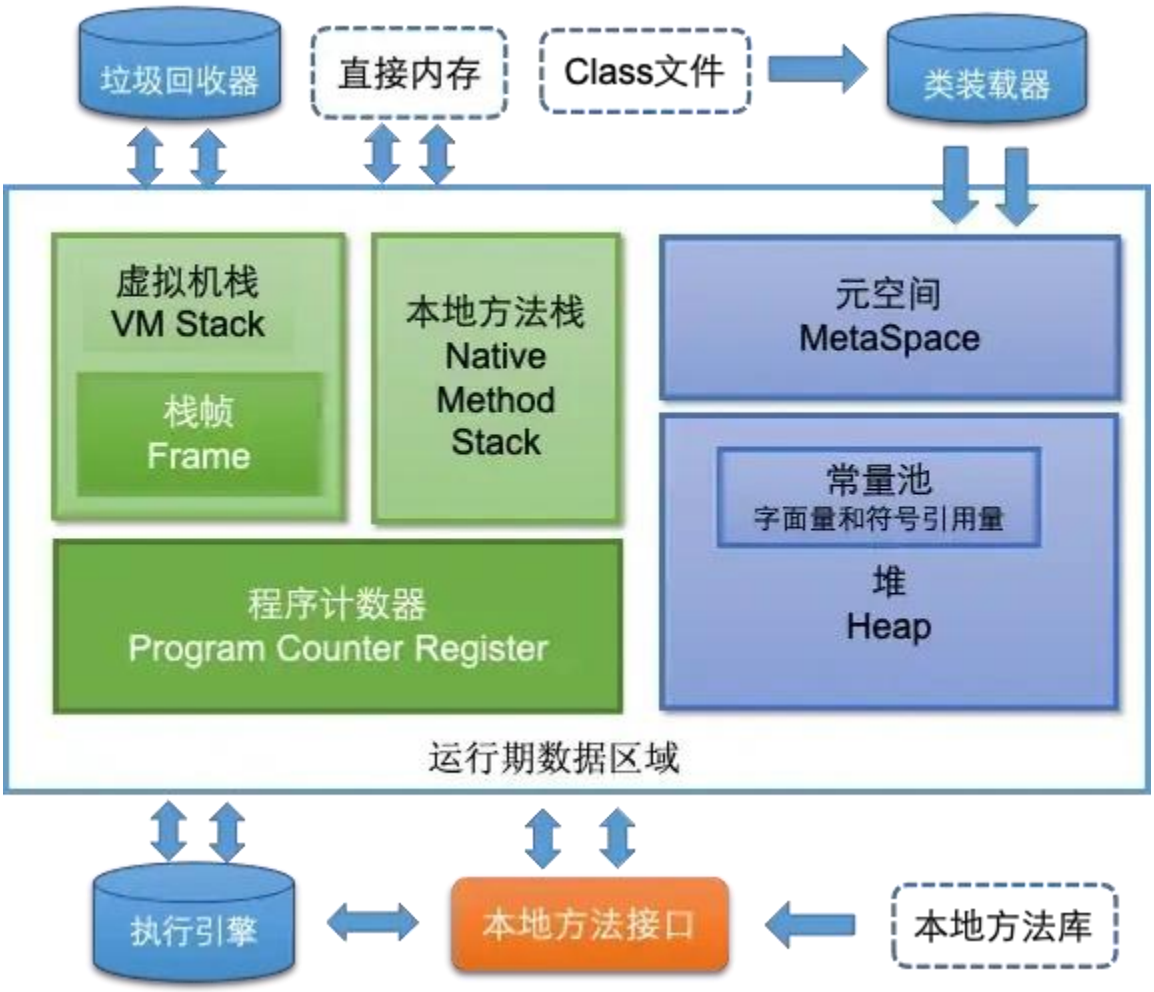
- **内存结构**：堆、栈、方法区、程序计数器各自存什么、有什么区别，堆里的新生代老年代怎么划分，这些是后面所有问题的基础。
- **垃圾回收**：可达性分析是怎么工作的、几种 GC 算法的原理和优缺点、Minor GC 和 Full GC 的触发条件，以及 CMS 和 G1 的区别，这条线是 JVM 面试的主线。
- **类加载机制**：类加载的过程（加载→验证→准备→解析→初始化）、双亲委派模型是什么、为什么要有双亲委派，这块问得也很频繁，逻辑理清楚之后其实不难答。
- **内存问题排查**：堆溢出和栈溢出的原因与解决思路、内存泄漏和内存溢出的区别，这类题在面试里越来越多，结合实际案例来答会比纯背理论效果好很多。

如果你是第一次系统准备 JVM，建议先把内存结构搞清楚，再去看垃圾回收，这两块是理解其他内容的前提，其他知识点理解起来也会顺很多。

内存模型

JVM 的内存模型介绍一下

根据 JDK 8 规范，JVM 运行时内存共分为虚拟机栈、堆、元空间、程序计数器、本地方法栈五个部分。还有一部分内存叫直接内存，属于操作系统的本地内存，也是可以直接操作的。



JVM 的内存结构主要分为以下几个部分：

- **程序计数器**：可以看作是当前线程所执行的字节码的行号指示器，用于存储当前线程正在执行的 Java 方法的 JVM 指令地址。如果线程执行的是 Native 方法，计数器值为 null。是唯一一个在 Java 虚拟机规范中没有规定任何 OutOfMemoryError 情况的区域，生命周期与线程相同。
- **Java 虚拟机栈**：每个线程都有自己独立的 Java 虚拟机栈，生命周期与线程相同。每个方法在执行时都会创建一个栈帧，用于存储局部变量表、操作数栈、动态链接、方法出口等信息。可能会抛出 StackOverflowError 和 OutOfMemoryError 异常。
- **本地方法栈**：与 Java 虚拟机栈类似，主要为虚拟机使用到的 Native 方法服务，在 HotSpot 虚拟机中和 Java 虚拟机栈合二为一。本地方法执行时也会创建栈帧，同样可能出现 StackOverflowError 和 OutOfMemoryError 两种错误。
- **Java 堆**：是 JVM 中最大的一块内存区域，被所有线程共享，在虚拟机启动时创建，用于存放对象实例。从内存回收角度，堆被划分为新生代和老年代，新生代又分为 Eden 区和两个 Survivor 区（From Survivor 和 To Survivor）。如果在堆中没有内存完成实例分配，并且堆也无法扩展时会抛出 OutOfMemoryError 异常。

- **方法区（元空间）**：在 JDK 1.8 及以后的版本中，方法区被元空间取代，使用本地内存。用于存储已被虚拟机加载的类信息、常量、静态变量等数据。虽然方法区被描述为堆的逻辑部分，但有 “非堆” 的别名。方法区可以选择不实现垃圾收集，内存不足时会抛出 OutOfMemoryError 异常。
- **运行时常量池**：是方法区的一部分，用于存放编译期生成的各种字面量和符号引用，具有动态性，运行时也可将新的常量放入池中。当无法申请到足够内存时，会抛出 OutOfMemoryError 异常。
- **直接内存**：不属于 JVM 运行时数据区的一部分，通过 NIO 类引入，是一种堆外内存，可以显著提高 I/O 性能。直接内存的使用受到本机总内存的限制，若分配不当，可能导致 OutOfMemoryError 异常。

JVM 内存模型里的堆和栈有什么区别?

- **用途**：栈主要用于存储局部变量、方法调用的参数、方法返回地址以及一些临时数据。每当一个方法被调用，一个栈帧（stack frame）就会在栈中创建，用于存储该方法的信息，当方法执行完毕，栈帧也会被移除。堆用于存储对象的实例（包括类的实例和数组）。当你使用 `new` 关键字创建一个对象时，对象的实例就会在堆上分配空间。
- **生命周期**：栈中的数据具有确定的生命周期，当一个方法调用结束时，其对应的栈帧就会被销毁，栈中存储的局部变量也会随之消失。堆中的对象生命周期不确定，对象会在垃圾回收机制（Garbage Collection, GC）检测到对象不再被引用时才被回收。
- **存取速度**：栈的存取速度通常比堆快，因为栈遵循先进后出（LIFO, Last In First Out）的原则，操作简单快速。堆的存取速度相对较慢，因为对象在堆上的分配和回收需要更多的时间，而且垃圾回收机制的运行也会影响性能。
- **存储空间**：栈的空间相对较小，且固定，由操作系统管理。当栈溢出时，通常是因为递归过深或局部变量过大。堆的空间较大，动态扩展，由 JVM 管理。堆溢出通常是由于创建了太多的大对象或未能及时回收不再使用的对象。
- **可见性**：栈中的数据对线程是私有的，每个线程有自己的栈空间。堆中的数据对线程是共享的，所有线程都可以访问堆上的对象。

栈中存的到底是指针还是对象?

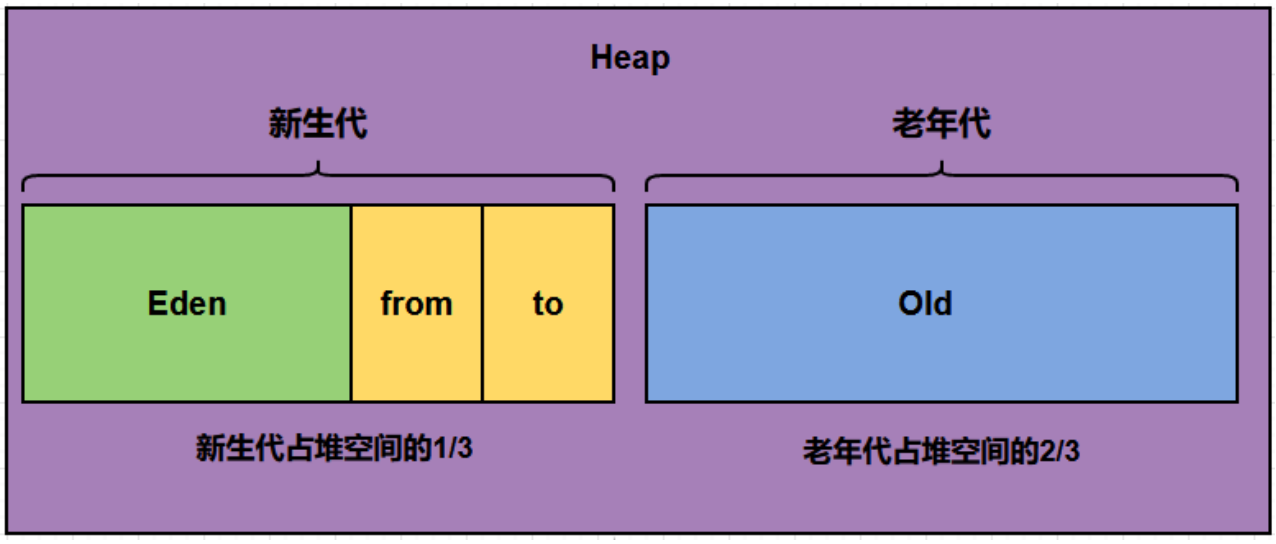
在 JVM 内存模型中，栈（Stack）主要用于管理线程的局部变量和方法调用的上下文，而堆（Heap）则是用于存储所有类的实例和数组。

当我们在栈中讨论“存储”时，实际上指的是存储基本类型的数据（如 int，double 等）和对象的引用，而不是对象本身。

这里的关键点是，栈中存储的**不是**对象，而是**对象的引用**。也就是说，当你在方法中声明一个对象，比如 `MyObject obj = new MyObject();`，这里的 `obj` 实际上是一个存储在栈上的引用，指向堆中实际的对象实例。这个引用是一个固定大小的数据（例如在 64 位系统上是 8 字节），它指向堆中分配给对象的内存区域。

堆分为哪几部分呢?

Java 堆（Heap）是 Java 虚拟机（JVM）中内存管理的一个重要区域，主要用于存放对象实例和数组。随着 JVM 的发展和不同垃圾收集器的实现，堆的具体划分可能会有所不同，但通常可以分为以下几个部分：



- **新生代（Young Generation）**:新生代分为 Eden Space 和 Survivor Space。在 Eden Space 中， 大多数新创建的对象首先存放在这里。Eden 区相对较小，当 Eden 区满时，会触发一次 Minor GC（新生代垃圾回收）。在 Survivor Spaces 中，通常分为两个相等大小的区域，称为 S0（Survivor 0）和 S1（Survivor 1）。在每次 Minor GC 后，存活下来的对象会被移动到其中一个 Survivor 空间，以继续它们的生命周期。这两个区域轮流充当对象的中转站，帮助区分短暂存活的对象和长期存活的对象。
- **老年代（Old Generation/Tenured Generation）**:存放过一次或多次 Minor GC 仍存活的对象会被移动到老年代。老年代中的对象生命周期较长，因此 Major GC（也称为 Full GC，涉及老年代的垃圾回收）发生的频率相对较低，但其执行时间通常比 Minor GC 长。老年代的空间通常比新生代大，以存储更多的长期存活对象。

- **元空间（Metaspace）**:从 Java 8 开始，永久代（Permanent Generation）被元空间取代，用于存储类的元数据信息，如类的结构信息（如字段、方法信息等）。元空间并不在 Java 堆中，而是使用本地内存，这解决了永久代容易出现的内存溢出问题。
- **大对象区（Large Object Space / Humongous Objects）**:在某些 JVM 实现中（如 G1 垃圾收集器），为大对象分配了专门的区域，称为大对象区或 Humongous Objects 区域。大对象是指需要大量连续内存空间的对象，如大数组。这类对象直接分配在老年代，以避免因频繁的年轻代晋升而导致的内存碎片化问题。

如果有个大对象一般是在哪个区域?

大对象通常会直接分配到老年代。

新生代主要用于存放生命周期较短的对象，并且其内存空间相对较小。如果将大对象分配到新生代，可能会很快导致新生代空间不足，从而频繁触发 Minor GC。而每次 Minor GC 都需要进行对象的复制和移动操作，这会带来一定的性能开销。将大对象直接分配到老年代，可以减少新生代的内存压力，降低 Minor GC 的频率。

大对象通常需要连续的内存空间，如果在新生代中频繁分配和回收大对象，容易产生内存碎片，导致后续分配大对象时可能因为内存不连续而失败。老年代的空间相对较大，更适合存储大对象，有助于减少内存碎片的产生。

程序计数器的作用，为什么是私有的?

Java 程序是支持多线程一起运行的，多个线程一起运行的时候 cpu 会有一个调动器组件给它们分配时间片，比如说会给线程 1 分给一个时间片，它在时间片内如果它的代码没有执行完，它就会把线程 1 的状态执行一个暂存，切换到线程 2 去，执行线程 2 的代码，等线程 2 的代码执行到了一定程度，线程 2 的时间片用完了，再切换回来，再继续执行线程 1 剩余部分的代码。

我们考虑一下，如果在线程切换的过程中，下一条指令执行到哪里了，是不是还是会用到我们的程序计数器啊。每个线程都有自己的程序计数器，因为它们各自执行的代码的指令地址是不一样的呀，所以每个线程都应该有自己的程序计数器。

方法区中的方法的执行过程?

当程序中通过对象或类直接调用某个方法时，主要包括以下几个步骤：

- **解析方法调用**：JVM 会根据方法的符号引用找到实际的方法地址（如果之前没有解析过的话）。

- **栈帧创建**：在调用一个方法前，JVM 会在当前线程的 Java 虚拟机栈中为该方法分配一个新的栈帧，用于存储局部变量表、操作数栈、动态链接、方法出口等信息。
- **执行方法**：执行方法内的字节码指令，涉及的操作可能包括局部变量的读写、操作数栈的操作、跳转控制、对象创建、方法调用等。
- **返回处理**：方法执行完毕后，可能会返回一个结果给调用者，并清理当前栈帧，恢复调用者的执行环境。

方法区中还有哪些东西?

《深入理解 Java 虚拟机》书中对方法区（Method Area）存储内容描述如下：它用于存储已被虚拟机加载的**类型信息、常量、静态变量、即时编译器编译后的代码缓存**等。

- 类信息：包括类的结构信息、类的访问修饰符、父类与接口等信息。
- 常量池：存储类和接口中的常量，包括字面值常量、符号引用，以及运行时常量池。
- 静态变量：存储类的静态变量，这些变量在类初始化的时候被赋值。
- 方法字节码：存储类的方法字节码，即编译后的代码。
- 符号引用：存储类和方法的符号引用，是一种直接引用不同于直接引用的引用类型。
- 运行时常量池：存储着在类文件中的常量池数据，在类加载后在方法区生成该运行时常量池。
- 常量池缓存：用于提升类加载的效率，将常用的常量缓存起来方便使用。

String 保存在哪里呢?

String 保存在字符串常量池中，不同于其他对象，它的值是不可变的，且可以被多个引用共享。

String s = new String（“abc”）执行过程中分别对应哪些内存区域?

首先，我们看到这个代码中有一个 new 关键字，我们知道 new 指令是创建一个类的实例对象并完成加载初始化的，因此这个字符串对象是在**运行期**才能确定的，创建的字符串对象是在**堆内存上**。

其次，在 String 的构造方法中传递了一个字符串 abc，由于这里的 abc 是被 final 修饰的属性，所以它是一个字符串常量。在首次构建这个对象时，JVM 拿字面量"abc"去字符串常量池试图获取其对应 String 对象的引用。于是在堆中创建了一个"abc"的 String 对象，并将其引用保存到字符串常量池中，然后返回；

所以，**如果 abc 这个字符串常量不存在，则创建两个对象，分别是 abc 这个字符串常量，以及 new String 这个实例对象。如果 abc 这字符串常量存在，则只会创建一个对象**。

引用类型有哪些？有什么区别？

引用类型主要分为强软弱虚四种：

- 强引用指的就是代码中普遍存在的赋值方式，比如 A a = new A()这种。强引用关联的对象，永远不会被 GC 回收。
- 软引用可以用 SoftReference 来描述，指的是那些有用但是不是必须要的对象。系统在发生内存溢出前会对这类引用的对象进行回收。
- 弱引用可以用 WeakReference 来描述，他的强度比软引用更低一点，弱引用的对象下一次 GC 的时候一定会被回收，而不管内存是否足够。
- 虚引用也被称作幻影引用，是最弱的引用关系，可以用 PhantomReference 来描述，他必须和 ReferenceQueue 一起使用，同样的当发生 GC 的时候，虚引用也会被回收。可以用虚引用来管理堆外内存。

弱引用了解吗?举例说明在哪里可以用?

Java 中的弱引用是一种引用类型，它不会阻止一个对象被垃圾回收。

在 Java 中，弱引用是通过 `Java.lang.ref.WeakReference` 类实现的。弱引用的一个主要用途是创建非强制性的对象引用，这些引用可以在内存压力大时被垃圾回收器清理，从而避免内存泄露。

弱引用的使用场景：

- **缓存系统**：弱引用常用于实现缓存，特别是当希望缓存项能够在内存压力下自动释放时。如果缓存的大小不受控制，可能会导致内存溢出。使用弱引用来维护缓存，可以让 JVM 在需要更多内存时自动清理这些缓存对象。
- **对象池**：在对象池中，弱引用可以用来管理那些暂时不使用的对象。当对象不再被强引用时，它们可以被垃圾回收，释放内存。
- **避免内存泄露**：当一个对象不应该被长期引用时，使用弱引用可以防止该对象被意外地保留，从而避免潜在的内存泄露。

示例代码：

假设我们有一个缓存系统，我们使用弱引用来维护缓存中的对象：

```
import Java.lang.ref.WeakReference;
import Java.util.HashMap;
import Java.util.Map;
```

```
public class CacheExample {

    private Map<String, WeakReference<MyHeavyObject>> cache = new HashMap<>();

    public MyHeavyObject get(String key) {
        WeakReference<MyHeavyObject> ref = cache.get(key);
        if (ref != null) {
            return ref.get();
        } else {
            MyHeavyObject obj = new MyHeavyObject();
            cache.put(key, new WeakReference<>(obj));
            return obj;
        }
    }

    // 假设 MyHeavyObject 是一个占用大量内存的对象
    private static class MyHeavyObject {
        private byte[] largeData = new byte[1024 * 1024 * 10]; // 10MB data
    }
}
```

在这个例子中，使用 `WeakReference` 来存储 `MyHeavyObject` 实例，当内存压力增大时，垃圾回收器可以自由地回收这些对象，而不会影响缓存的正常运行。

如果一个对象被垃圾回收，下次尝试从缓存中获取时，`get()`方法会返回 `null`，这时我们可以重新创建对象并将其放入缓存中。因此，使用弱引用时要注意，一旦对象被垃圾回收，通过弱引用获取的对象可能会变为 `null`，因此在使用前通常需要检查这一点。

内存泄漏和内存溢出的理解?

内存泄露：内存泄漏是指程序在运行过程中不再使用的对象仍然被引用，而无法被垃圾收集器回收，从而导致可用内存逐渐减少。虽然在 Java 中，垃圾回收机制会自动回收不再使用的对象，但如果有对象仍被不再使用的引用持有，垃圾收集器无法回收这些内存，最终可能导致程序的内存使用不断增加。

内存泄露常见原因：

- **静态集合**：使用静态数据结构（如 `HashMap` 或 `ArrayList`）存储对象，且未清理。

- **事件监听**：未取消对事件源的监听，导致对象持续被引用。
- **线程**：未停止的线程可能持有对象引用，无法被回收。

内存溢出：内存溢出是指 Java 虚拟机（JVM）在申请内存时，无法找到足够的内存，最终引发 `OutOfMemoryError`。这通常发生在堆内存不足以存放新创建的对象时。

内存溢出常见原因：

- **大量对象创建**：程序中不断创建大量对象，超出 JVM 堆的限制。
- **持久引用**：大型数据结构（如缓存、集合等）长时间持有对象引用，导致内存累积。
- **递归调用**：深度递归导致栈溢出。

jvm 内存结构有哪几种内存溢出的情况?

- **堆内存溢出**：当出现 Java.lang.OutOfMemoryError:Java heap space 异常时，就是堆内存溢出了。原因是代码中可能存在大对象分配，或者发生了内存泄露，导致在多次 GC 之后，还是无法找到一块足够大的内存容纳当前对象。
- **栈溢出**：如果我们写一段程序不断的进行递归调用，而且没有退出条件，就会导致不断地进行压栈。类似这种情况，JVM 实际会抛出 StackOverFlowError；当然，如果 JVM 试图去扩展栈空间的的时候失败，则会抛出 OutOfMemoryError。
- **元空间溢出**：元空间的溢出，系统会抛出 Java.lang.OutOfMemoryError: Metaspace。出现这个异常的问题的原因是系统的代码非常多或引用的第三方包非常多或者通过动态代码生成类加载等方法，导致元空间的内存占用很大。
- **直接内存内存溢出**：在使用 ByteBuffer 中的 allocateDirect()的时候会用到，很多 JavaNIO(像 netty)的框架中被封装为其他的方法，出现该问题时会抛出 Java.lang.OutOfMemoryError: Direct buffer memory 异常。

遇到过堆溢出的情况吗？如何解决?

堆溢出（`java.lang.OutOfMemoryError: Java heap space`）通常发生在程序持续创建对象且无法被 GC 及时回收的场景下。

遇到堆溢出时，首先需要定位原因，一般分两步：

1. **捕获内存快照**：通过 JVM 参数`-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=./heapdump.hprof`，让程序在发生 OOM 时自动生成堆快照文件。

2. **分析快照文件**：使用 MAT（Memory Analyzer Tool）或 JProfiler 等工具分析快照，重点看哪些对象占用了大量内存、是否存在内存泄漏（如对象长期被无用引用持有，无法回收）。

常见的解决思路根据原因不同而不同：

- **如果是内存泄漏**：比如静态集合无意识地缓存了大量对象、长生命周期对象持有短生命周期对象的引用（如单例类持有业务对象）等。这时候需要梳理对象引用链，找到未释放的根源，比如清理静态集合中不再使用的元素、解除不必要的对象关联。
- **如果是内存不足**：即程序确实需要大量内存（如处理大文件、加载大量数据到内存），但当前堆配置太小。这种情况下可以通过调整 JVM 参数扩大堆内存，比如`-Xms2g -Xmx4g`（初始堆 2G，最大堆 4G），但需注意不能超过物理内存限制，避免频繁 swap。

另外，从代码层面优化也很重要：比如避免一次性加载全部数据（改用分批处理）、使用缓存时设置合理的过期策略、及时释放资源（如 IO 流、数据库连接）等，从源头减少内存占用。

举个例子：曾遇到过一个批量处理任务，代码中把所有处理结果都存到了一个静态 List 里，导致对象越积越多，最终堆溢出。通过分析快照发现这个 List 占用了 80% 的堆内存，修改为处理完一批就写入数据库并清空 List 后，问题解决。

栈溢出的情况呢?

栈溢出（`java.lang.StackOverflowError`）是 JVM 中另一种常见的内存错误，和堆溢出的原理与场景截然不同。栈溢出主要发生在 Java 虚拟机栈（或本地方法栈）的内存空间被耗尽时，通常与方法调用的深度直接相关。

从触发原因来看，最常见的场景是**无限递归调用**。因为 Java 方法调用时会在栈中创建栈帧（存储局部变量、操作数栈、方法返回地址等），每递归一次就会新增一个栈帧。如果递归没有正确的终止条件，栈帧会不断累积，最终超过虚拟机栈的最大容量，导致栈溢出。比如一个简单的无终止条件的递归方法：

```
public void recursiveMethod() {
    recursiveMethod(); // 无限递归，没有终止条件
}
```

调用这个方法很快就会抛出`StackOverflowError`。

另一种情况是**单个方法的栈帧过大**。如果一个方法定义了大量局部变量，或者局部变量占用内存过大（比如大数组），单个栈帧就会占用较多栈空间，可能在调用层级不深时就耗尽栈内存。

解决栈溢出的思路主要有：

1. **排查递归逻辑**：检查是否存在无限递归或递归层级过深的问题，添加正确的终止条件，或减少递归深度。必要时可将递归改写为迭代（如用循环替代），因为迭代不会持续创建新栈帧。
2. **调整栈内存大小**：通过 JVM 参数`-Xss`（如`-Xss256k`）增大栈内存容量。但这种方式要谨慎，栈内存过大会导致线程可创建数量减少（总内存固定时，单个线程栈越大，能创建的线程数越少）。
3. **优化方法栈帧**：减少方法内局部变量的数量，避免在方法中创建过大的对象或数组，将大对象的创建移到堆中（通过 new 关键字），降低单个栈帧的内存占用。

举个实际例子：曾遇到一个树形结构遍历的方法，因节点层级极深（超过 10 万层）且用递归实现，导致栈溢出。解决方式是将递归遍历改为基于栈的迭代遍历，手动维护节点访问顺序，避免了栈帧的无限累积，问题得以解决。

总的来说，栈溢出的核心原因是方法调用栈深度超过了栈内存限制，解决时应优先从代码逻辑（尤其是递归）入手，而非单纯调大栈内存。

有具体的内存泄漏和内存溢出的例子么请举例及解决方案?

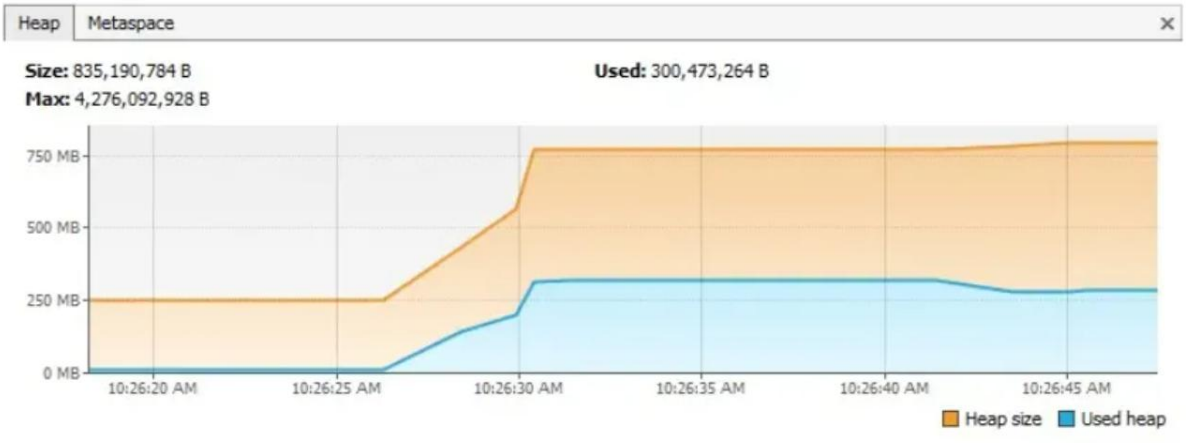
> 1、静态属性导致内存泄露

会导致内存泄露的一种情况就是大量使用 static 静态变量。在 Java 中，静态属性的生命周期通常伴随着应用整个生命周期（除非 ClassLoader 符合垃圾回收的条件）。下面来看一个具体的会导致内存泄露的实例：

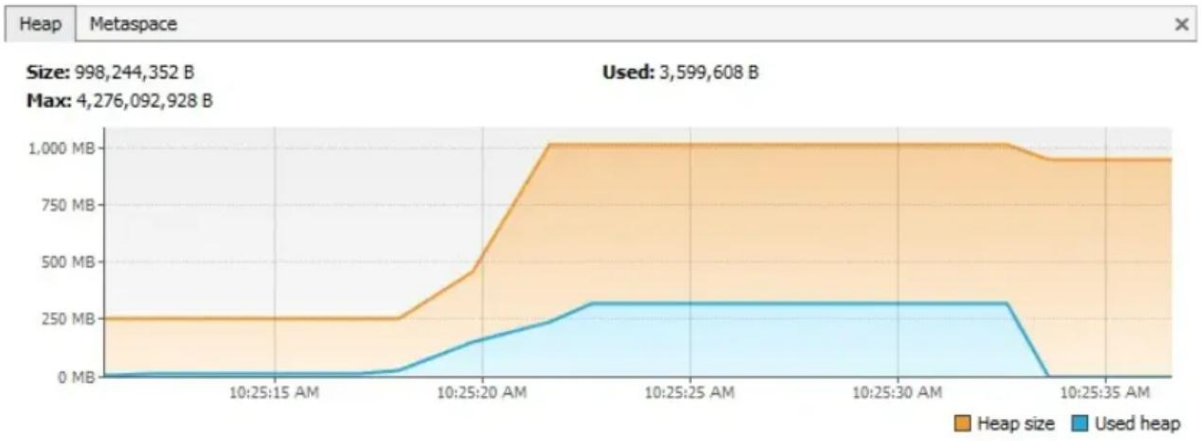
```
public class StaticTest {
    public static List<Double> list = new ArrayList<>();
    public void populateList() {
        for (int i = 0; i < 10000000; i++) {
            list.add(Math.random());
        }
        Log.info("Debug Point 2");
    }
    public static void main(String[] args) {
        Log.info("Debug Point 1");
        new StaticTest().populateList();
        Log.info("Debug Point 3");
    }
```

```
}
```

如果监控内存堆内存的变化，会发现在打印 Point1 和 Point2 之间，堆内存会有一个明显的增长趋势图。但当执行完 populateList 方法之后，对堆内存并没有被垃圾回收器进行回收。



但针对上述程序，如果将定义 list 的变量前的 static 关键字去掉，再次执行程序，会发现内存发生了具体的变化。VisualVM 监控信息如下图：



对比两个图可以看出，程序执行的前半部分内存使用情况都一样，但当执行完 populateList 方法之后，后者不再有引用指向对应的数据，垃圾回收器便进行了回收操作。因此，我们要十分留意 static 的变量，如果集合或大量的对象定义为 static 的，它们会停留在整个应用程序的生命周期当中。而它们所占用的内存空间，本可以用于其他地方。

那么如何优化呢？第一，进来减少静态变量；第二，如果使用单例，尽量采用懒加载。

2、 未关闭的资源

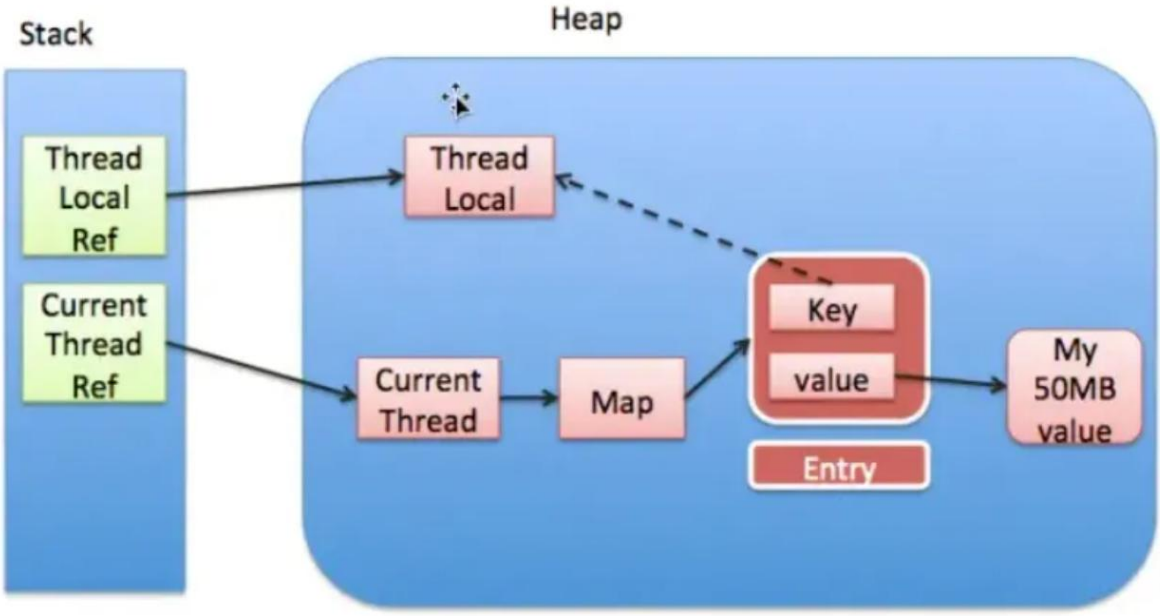
无论什么时候当我们创建一个连接或打开一个流，JVM 都会分配内存给这些资源。比如，数据库链接、输入流和 session 对象。

忘记关闭这些资源，会阻塞内存，从而导致 GC 无法进行清理。特别是当程序发生异常时，没有在 finally 中进行资源关闭的情况。这些未正常关闭的连接，如果不进行处理，轻则影响程序性能，重则导致 OutOfMemoryError 异常发生。

如果进行处理呢？第一，始终记得在 finally 中进行资源的关闭；第二，关闭连接的自身代码不能发生异常；第三，Java7 以上版本可使用 try-with-resources 代码方式进行资源关闭。

3、 使用 ThreadLocal

ThreadLocal 提供了线程本地变量，它可以保证访问到的变量属于当前线程，每个线程都保存有一个变量副本，每个线程的变量都不同。ThreadLocal 相当于提供了一种线程隔离，将变量与线程相绑定，从而实现线程安全的特性。



ThreadLocal 的实现中，每个 Thread 维护一个 ThreadLocalMap 映射表，key 是 ThreadLocal 实例本身，value 是真正需要存储的 Object。

ThreadLocalMap 使用 ThreadLocal 的弱引用作为 key，如果一个 ThreadLocal 没有外部强引用来引用它，那么系统 GC 时，这个 ThreadLocal 势必会被回收，这样一来，ThreadLocalMap 中就会出现 key 为 null 的 Entry，就没有办法访问这些 key 为 null 的 Entry 的 value。

如果当前线程迟迟不结束的话，这些 key 为 null 的 Entry 的 value 就会一直存在一条强引用链：Thread Ref -> Thread -> ThreaLocalMap -> Entry -> value 永远无法回收，造成内存泄漏。

如何解决此问题？

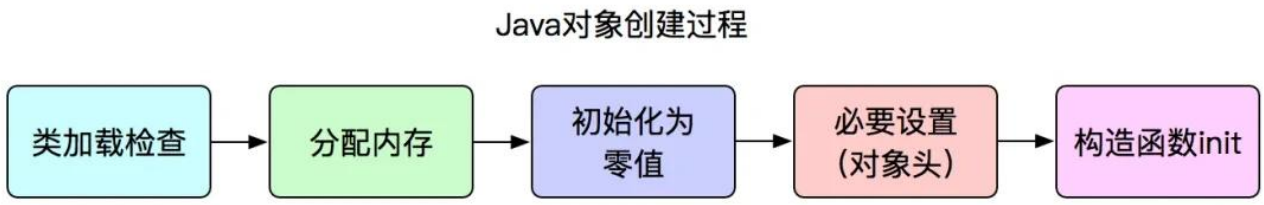
- 第一，使用 ThreadLocal 提供的 remove 方法，可对当前线程中的 value 值进行移除；

- 第二，不要使用 ThreadLocal.set(null) 的方式清除 value，它实际上并没有清除值，而是查找与当前线程关联的 Map 并将键值对分别设置为当前线程和 null。
- 第三，最好将 ThreadLocal 视为需要在 finally 块中关闭的资源，以确保即使在发生异常的情况下也始终关闭该资源。

```
try {
    threadLocal.set(System.nanoTime());
    //... further processing
} finally {
    threadLocal.remove();
}
```

类初始化和类加载

创建对象的过程?



在 Java 中创建对象的过程包括以下几个步骤：

1. **类加载检查**：虚拟机遇到一条 new 指令时，首先将去检查这个指令的参数是否能在**常量池**中定位到一个类的**符号引用**，并且检查这个符号引用代表的类是否已被**加载过、解析和初始化**过。如果没有，那必须先执行相应的**类加载过程**。
2. **分配内存**：在类加载检查通过后，接下来虚拟机将为新生对象分配内存。对象所需的**内存大小**在**类加载**完成后便可确定，为对象分配空间的任务等同于把一块确定大小的内存从 Java 堆中划分出来。
3. **初始化零值**：内存分配完成后，虚拟机需要将分配到的内存空间都初始化为零值（不包括对象头），这一步操作保证了对象的实例字段在 Java 代码中可以不赋初始值就直接使用，程序能访问到这些字段的数据类型所对应的零值。
4. **进行必要设置，比如对象头**：初始化零值完成之后，虚拟机要对对象进行**必要的设置**，例如这个对象是哪个类的实例、如何才能找到类的元数据信息、对象的哈希码、对象的 GC 分代

年龄等信息。这些信息存放在**对象头**中。另外，根据虚拟机当前运行状态的不同，如是否启用偏向锁等，对象头会有不同的设置方式。

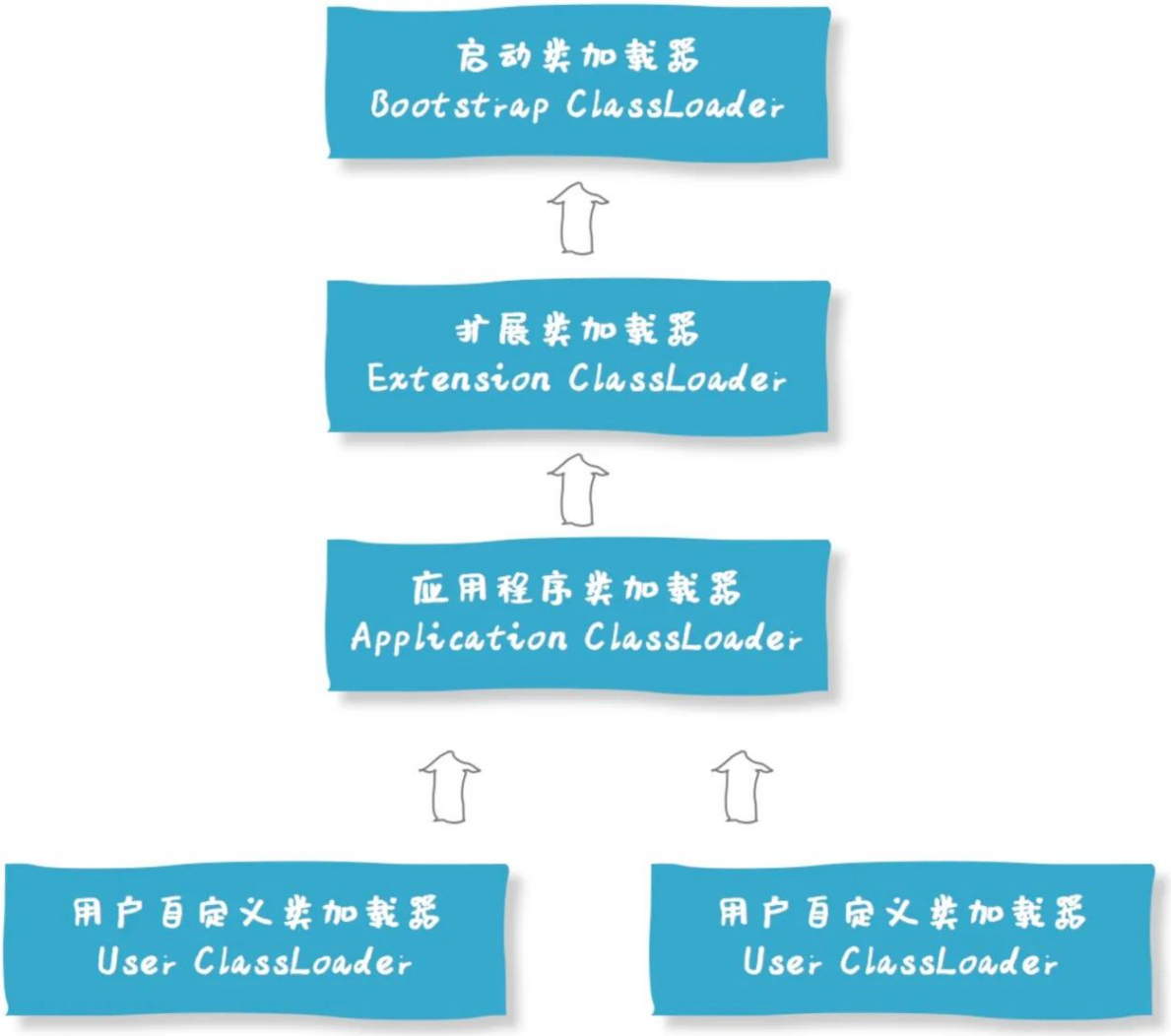
5. **执行 init 方法**：在上面工作都完成之后，从虚拟机的视角来看，一个新的对象已经产生了，但从 Java 程序的视角来看，对象创建才刚开始——构造函数，即 class 文件中的方法还没有执行，所有的字段都还为零，**对象需要的其他资源和状态信息**还没有按照预定的意图构造好。所以一般来说，执行 new 指令之后会接着执行方法，把对象按照程序员的意愿进行初始化，这样一个真正可用的对象才算完全被构造出来。

对象的生命周期

对象的生命周期包括创建、使用和销毁三个阶段：

- 创建：对象通过关键字 new 在堆内存中被实例化，构造函数被调用，对象的内存空间被分配。
- 使用：对象被引用并执行相应的操作，可以通过引用访问对象的属性和方法，在程序运行过程中被不断使用。
- 销毁：当对象不再被引用时，通过垃圾回收机制自动回收对象所占用的内存空间。垃圾回收器会在适当的时候检测并回收不再被引用的对象，释放对象占用的内存空间，完成对象的销毁过程。

类加载器有哪些?



- **启动类加载器（Bootstrap Class Loader）**：这是最顶层的类加载器，负责加载 Java 的核心库（如位于 jre/lib/rt.jar 中的类），它是用 C++编写的，是 JVM 的一部分。启动类加载器无法被 Java 程序直接引用。
- **扩展类加载器（Extension Class Loader）**：它是 Java 语言实现的，继承自 ClassLoader 类，负责加载 Java 扩展目录（jre/lib/ext 或由系统变量 Java.ext.dirs 指定的目录）下的 jar 包和类库。扩展类加载器由启动类加载器加载，并且父加载器就是启动类加载器。
- **系统类加载器（System Class Loader）/ 应用程序类加载器（Application Class Loader）**：这也是 Java 语言实现的，负责加载用户类路径（ClassPath）上的指定类库，是我们平时编写 Java 程序时默认使用的类加载器。系统类加载器的父加载器是扩展类加载器。它可以通过 ClassLoader.getSystemClassLoader()方法获取到。
- **自定义类加载器（Custom Class Loader）**：开发者可以根据需求定制类的加载方式，比如从网络加载 class 文件、数据库、甚至是加密的文件中加载类等。自定义类加载器可以用来扩展 Java 应用程序的灵活性和安全性，是 Java 动态性的一个重要体现。

这些类加载器之间的关系形成了双亲委派模型，其核心思想是当一个类加载器收到类加载的请求时，首先不会自己去尝试加载这个类，而是把这个请求委派给父类加载器去完成，每一层次的类加载器都是如此，因此所有的加载请求最终都应该传送到顶层的启动类加载器中。

只有当父加载器反馈自己无法完成这个加载请求（它的搜索范围中没有找到所需的类）时，子加载器才会尝试自己去加载。

Java 中 双亲委派 是什么？有啥用?

Java 中的 “双亲委派” 是类加载机制的核心原则，简单说就是 「**一个类加载器要加载类时，先让父加载器去尝试加载，只有父加载器加载不了，自己才会去加载**」。这里的 “双亲” 并不是指真正的继承关系，而是类加载器之间的一种层级委派关系。

具体来说，Java 的类加载器有一套默认的层级结构：最顶层是 `Bootstrap ClassLoader`（启动类加载器，负责加载 JDK 核心类，如 `java.lang.String`），往下是 `Extension ClassLoader`（扩展类加载器，加载 JDK 扩展目录的类），再往下是 `AppClassLoader`（应用类加载器，加载我们自己写的类和第三方 jar 包），我们也可以自定义类加载器，放在最下层。

当某个类加载器（比如自定义加载器）收到加载类的请求时，它不会先自己动手，而是把请求 “委派” 给父加载器；父加载器同样会继续委派给它的父加载器，直到传到最顶层的启动类加载器。如果父加载器能找到并加载这个类，就直接返回；如果所有父加载器都加载不了（比如不在它们的加载范围内），子加载器才会自己去尝试加载。

举个例子：我们自己写了一个 `java.lang.String` 类，当 `AppClassLoader` 要加载它时，会先委派给 `Extension ClassLoader`，再委派给 `Bootstrap ClassLoader`。而启动类加载器发现自己已经加载过 JDK 自带的 `String` 类了，就直接返回这个类，不会去加载我们自定义的 `String` 类。

这种机制的核心作用有两个：

1. **保证类的唯一性和安全性**：避免同一个类被不同加载器重复加载，确保核心类（如 JDK 的 `String`、`Integer`）不会被篡改。比如上面的例子，防止我们自定义的 `String` 类替换掉 JDK 的核心类，否则可能引发安全问题（比如修改 `String` 的底层实现导致系统混乱）。
2. **实现类的复用**：核心类只需要被顶层加载器加载一次，所有子加载器都能共享这个类，减少内存消耗。

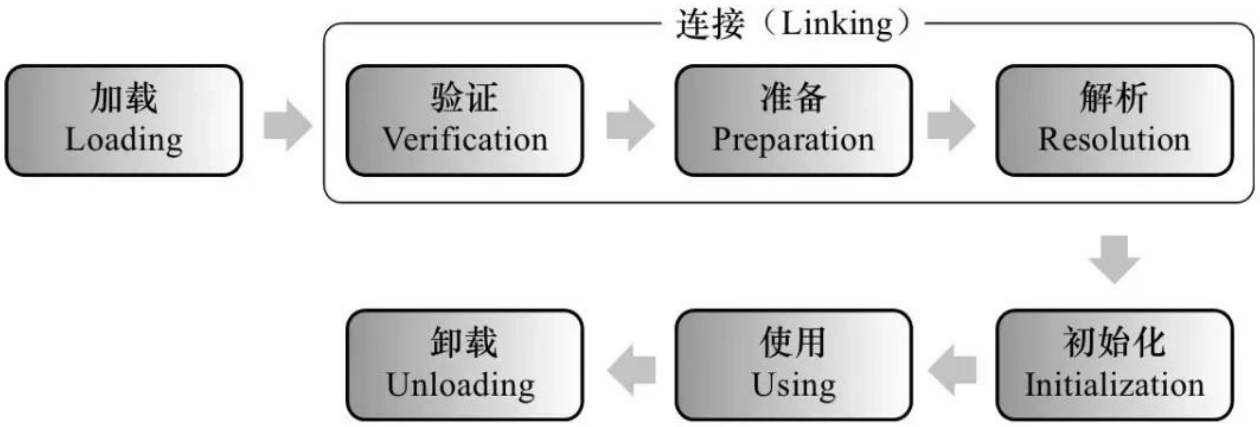
简单说，双亲委派就像 「孩子找东西先问家长，家长解决不了再自己找」，通过层级委派确保了 Java 核心类的安全和类加载的有序性，是 Java 运行时环境稳定的基础。

双亲委派模型的作用

- **保证类的唯一性**：通过委托机制，确保了所有加载请求都会传递到启动类加载器，避免了不同类加载器重复加载相同类的情况，保证了 Java 核心类库的统一性，也防止了用户自定义类覆盖核心类库的可能。
- **保证安全性**：由于 Java 核心库被启动类加载器加载，而启动类加载器只加载信任的类路径中的类，这样可以防止不可信的类假冒核心类，增强了系统的安全性。例如，恶意代码无法自定义一个 Java.lang.System 类并加载到 JVM 中，因为这个请求会被委托给启动类加载器，而启动类加载器只会加载标准的 Java 库中的类。
- **支持隔离和层次划分**：双亲委派模型支持不同层次的类加载器服务于不同的类加载需求，如应用程序类加载器加载用户代码，扩展类加载器加载扩展框架，启动类加载器加载核心库。这种层次化的划分有助于实现沙箱安全机制，保证了各个层级类加载器的职责清晰，也便于维护和扩展。
- **简化了加载流程**：通过委派，大部分类能够被正确的类加载器加载，减少了每个加载器需要处理的类的数量，简化了类的加载过程，提高了加载效率。

讲一下类加载过程?

类从被加载到虚拟机内存开始，到卸载出内存为止，它的整个生命周期包括以下 7 个阶段：



- **加载**：通过类的全限定名（包名 + 类名），获取到该类的.class 文件的二进制字节流，将二进制字节流所代表的静态存储结构，转化为方法区运行时的数据结构，在内存中生成一个代表该类的 Java.lang.Class 对象，作为方法区这个类的各种数据的访问入口
- **连接**：验证、准备、解析 3 个阶段统称为连接。
    - **验证**：确保 class 文件中的字节流包含的信息，符合当前虚拟机的要求，保证这个被加载的 class 类的正确性，不会危害到虚拟机的安全。验证阶段大致会完成以下四个阶段的检验动作：文件格式校验、元数据验证、字节码验证、符号引用验证

    - **准备**：为类中的静态字段分配内存，并设置默认的初始值，比如 int 类型初始值是 0。被 final 修饰的 static 字段不会设置，因为 final 在编译的时候就分配了
    - **解析**：解析阶段是虚拟机将常量池的「符号引用」直接替换为「直接引用」的过程。符号引用是以一组符号来描述所引用的目标，符号可以是任何形式的字面量，只要使用的时候可以无歧义地定位到目标即可。直接引用可以是直接指向目标的指针、相对偏移量或是一个能间接定位到目标的句柄，直接引用是和虚拟机实现的内存布局相关的。如果有了直接引用， 那引用的目标必定已经存在在内存中了。
- **初始化**：初始化是整个类加载过程的最后一个阶段，初始化阶段简单来说就是执行类的构造器方法（() ），要注意的是这里的构造器方法()并不是开发者写的，而是编译器自动生成的。
- **使用**：使用类或者创建对象
- **卸载**：一个类要被 JVM 卸载，条件非常苛刻，**需要同时满足以下三点**：
    - **该类所有的实例都已经被回收**：这是最显而易见的前提。如果堆中还存在这个类的任何一个实例对象，那么定义这个对象的 Class 对象肯定不能被卸载。
    - **加载该类的 ClassLoader 已经被回收**：这是**最关键也是最难满足的条件**。类与其加载器是双向绑定的共生关系。一个类由哪个类加载器加载，这个信息是存储在 Class 对象里的。要卸载一个类，必须先卸载加载它的类加载器。
    - **类对应的 Java.lang.Class 对象没有任何地方被引用**：不能在任何地方通过反射（如静态字段、全局变量）、静态变量、JNI 等途径引用到这个 Class 对象。一旦这个 Class 对象还存在强引用，GC 就不会回收它，那么这个类也就不会被卸载。

讲一下类的加载和双亲委派原则

我们把 Java 的类加载过程分为三个主要步骤：加载、链接、初始化。

首先是加载阶段（Loading），它是 Java 将字节码数据从不同的数据源读取到 JVM 中，并映射为 JVM 认可的数据结构（Class 对象），这里的数据源可能是各种各样的形态，如 jar 文件、class 文件，甚至是网络数据源等；如果输入数据不是 ClassFile 的结构，则会抛出 ClassFormatError。

加载阶段是用户参与的阶段，我们可以自定义类加载器，去实现自己的类加载过程。

第二阶段是链接（Linking），这是核心的步骤，简单说是把原始的类定义信息平滑地转化入 JVM 运行的过程中。这里可进一步细分为三个步骤：

- 验证（Verification），这是虚拟机安全的重要保障，JVM 需要核验字节信息是符合 Java 虚拟机规范的，否则就被认为是 VerifyError，这样就防止了恶意信息或者不合规的信息危害 JVM 的运行，验证阶段有可能触发更多 class 的加载。
- 准备（Preparation），创建类或接口中的静态变量，并初始化静态变量的初始值。但这里的“初始化”和下面的显式初始化阶段是有区别的，侧重点在于分配所需要的内存空间，不会去执行更进一步的 JVM 指令。
- 解析（Resolution），在这一步会将常量池中的符号引用（symbolic reference）替换为直接引用。

最后是初始化阶段（initialization），这一步真正去执行类初始化的代码逻辑，包括静态字段赋值的动作，以及执行类定义中的静态初始化块内的逻辑，编译器在编译阶段就会把这部分逻辑整理好，父类型的初始化逻辑优先于当前类型的逻辑。

再来谈谈双亲委派模型，简单说就是当类加载器（Class-Loader）试图加载某个类型的时候，除非父加载器找不到相应类型，否则尽量将这个任务代理给当前加载器的父加载器去做。使用委派模型的目的是避免重复加载 Java 类型。

垃圾回收

什么是 Java 里的垃圾回收？如何触发垃圾回收？

垃圾回收（Garbage Collection, GC）是自动管理内存的一种机制，它负责自动释放不再被程序引用的对象所占用的内存，这种机制减少了内存泄漏和内存管理错误的可能性。垃圾回收可以通过多种方式触发，具体如下：

- **内存不足时**：当 JVM 检测到堆内存不足，无法为新的对象分配内存时，会自动触发垃圾回收。
- **手动请求**：虽然垃圾回收是自动的，开发者可以通过调用 `System.gc()` 或 `Runtime.getRuntime().gc()` 建议 JVM 进行垃圾回收。不过这只是一个建议，并不能保证立即执行。
- **JVM 参数**：启动 Java 应用时可以通过 JVM 参数来调整垃圾回收的行为，比如：`-Xmx`（最大堆大小）、`-Xms`（初始堆大小）等。
- **对象数量或内存使用达到阈值**：垃圾收集器内部实现了一些策略，以监控对象的创建和内存使用，达到某个阈值时触发垃圾回收。

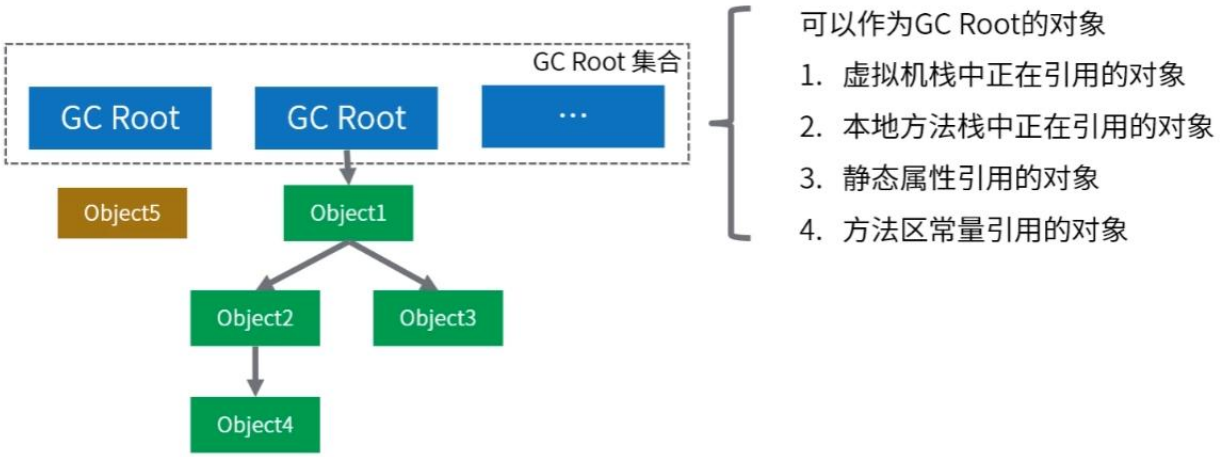
判断垃圾的方法有哪些？

在 Java 中，判断对象是否为垃圾（即不再被使用，可以被垃圾回收器回收）主要依据两种主流的垃圾回收算法来实现：**引用计数法和可达性分析算法**。

> 引用计数法（Reference Counting）

- **原理**：为每个对象分配一个引用计数器，每当有一个地方引用它时，计数器加 1；当引用失效时，计数器减 1。当计数器为 0 时，表示对象不再被任何变量引用，可以被回收。
- **缺点**：不能解决循环引用的问题，即两个对象相互引用，但不再被其他任何对象引用，这时引用计数器不会为 0，导致对象无法被回收。

> 可达性分析算法（Reachability Analysis）



Java 虚拟机主要采用此算法来判断对象是否为垃圾。

- **原理**：从一组称为 GC Roots（垃圾收集根）的对象出发，向下追溯它们引用的对象，以及这些对象引用的其他对象，以此类推。如果一个对象到 GC Roots 没有任何引用链相连（即从 GC Roots 到这个对象不可达），那么这个对象就被认为是不可达的，可以被回收。GC Roots 对象包括：虚拟机栈（栈帧中的本地变量表）中引用的对象、方法区中类静态属性引用的对象、本地方法栈中 JNI（Java Native Interface）引用的对象、活跃线程的引用等。

垃圾回收算法是什么，是为了解决了什么问题?

JVM 有垃圾回收机制的原因是为了解决内存管理的问题。在传统的编程语言中，开发人员需要手动分配和释放内存，这可能导致内存泄漏、内存溢出等问题。而 Java 作为一种高级语言，旨在提供更简单、更安全的编程环境，因此引入了垃圾回收机制来自动管理内存。

垃圾回收机制的主要目标是**自动检测和回收**不再使用的对象，从而释放它们所占用的内存空间。这样可以避免内存泄漏（一些对象被分配了内存却无法被释放，导致内存资源的浪费）。同时，垃圾回收机制还可以防止内存溢出（即程序需要的内存超过了可用内存的情况）。

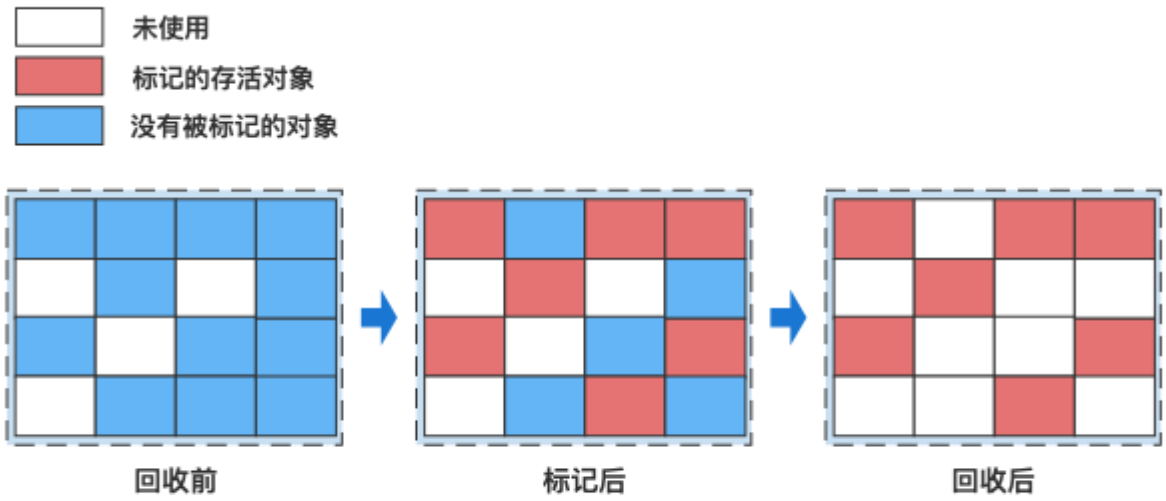
通过垃圾回收机制，JVM 可以在程序运行时自动识别和清理不再使用的对象，使得开发人员无需手动管理内存。这样可以提高开发效率、减少错误，并且使程序更加可靠和稳定。

java 的 gc 机制你知道哪些?

Java 的 GC（垃圾回收）简单说就是 「自动识别并回收不再使用的对象，释放内存空间」，避免内存泄漏。主要可以从 “哪些对象会被回收”“怎么回收”“回收器有哪些” 这几个角度来理解。

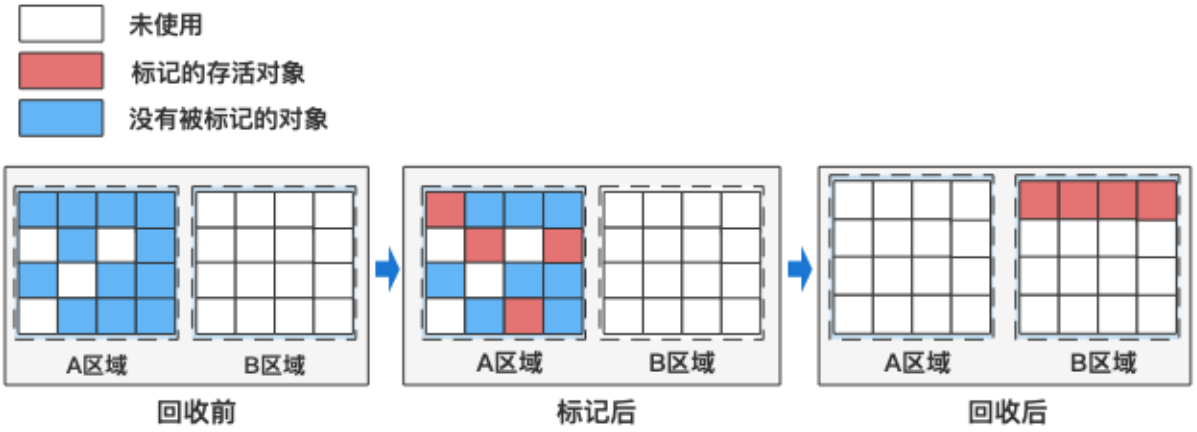
首先，判断一个对象是否该被回收，核心是看它 「是否还被引用」。最经典的是 「可达性分析」：以 GC Roots（比如虚拟机栈里的局部变量、静态变量、常量等）为起点，遍历对象的引用链，如果某个对象从 GC Roots 出发不可达，就会被标记为可回收对象。比如方法里的局部变量，方法执行完后出栈，它引用的对象如果没有其他引用，就会被标记。

然后是具体的回收算法。最基础的有 “标记 - 清除”：先标记所有可回收对象，然后直接清理它们的内存。但这样会产生很多内存碎片，后续分配大对象时可能找不到连续空间。



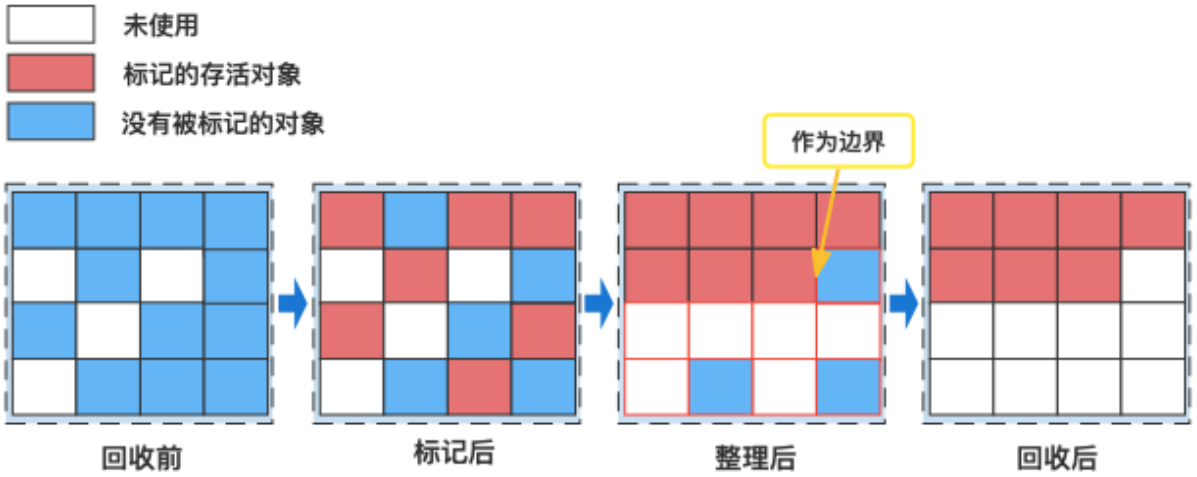
标记清除算法

为了解决碎片问题，有 “标记 - 整理”：标记后不是直接清除，而是把存活对象往一端移动，然后清理边界外的内存，这样内存更整齐，但移动对象会有性能开销。



复制算法

还有 “复制算法”：把内存分成两块，每次只用其中一块，回收时把存活对象复制到另一块，然后清空当前块。这种方式没有碎片，但会浪费一半内存，适合存活对象少的场景（比如新生代）。



标记整理算法

实际中，JVM 会分代使用这些算法。因为对象的生命周期不同，新生代（刚创建的对象）存活率低，老年代（存活久的对象）存活率高。新生代一般用 “复制算法”：分成 Eden 区和两个 Survivor 区（比如 8:1:1），新对象先放 Eden，满了就触发 Minor GC，把存活对象复制到一个 Survivor，多次存活后移到老年代。老年代用 “标记 - 清除” 或 “标记 - 整理”，因为对象存活率高，复制成本太高。

具体的垃圾回收器就是这些算法的实现，不同回收器适用场景不同。比如

- Serial GC 是单线程回收，简单但效率低，适合小内存应用；
- Parallel GC 多线程回收，注重吞吐量（单位时间内处理任务多）；

- CMS 追求低延迟，回收时大部分工作和用户线程并发执行，减少停顿时间，但会占用更多 CPU；
- G1 则把内存分成多个区域，优先回收垃圾多的区域，平衡吞吐量和延迟，适合大内存场景。
- 现在新出的 ZGC、Shenandoah 更是追求微秒级停顿，适合高并发低延迟的应用。

总的来说，GC 机制的核心是 「自动管理内存」，通过可达性分析判断垃圾，结合分代思想和不同算法，再由具体的回收器实现，最终目的是在保证程序正常运行的前提下，高效释放内存，减少对用户线程的影响。

垃圾回收算法有哪些?

- **标记-清除算法**：标记-清除算法分为“标记”和“清除”两个阶段，首先通过可达性分析，标记出所有需要回收的对象，然后统一回收所有被标记的对象。标记-清除算法有两个缺陷，一个是效率问题，标记和清除的过程效率都不高，另外一个就是，清除结束后会造成大量的碎片空间。有可能会造成在申请大块内存的时候因为没有足够的连续空间导致再次 GC。
- **复制算法**：为了解决碎片空间的问题，出现了“复制算法”。复制算法的原理是，将内存分成两块，每次申请内存时都使用其中的一块，当内存不够时，将这一块内存中所有存活的复制到另一块上。然后将然后再把已使用的内存整个清理掉。复制算法解决了空间碎片的问题。但是也带来了新的问题。因为每次在申请内存时，都只能使用一半的内存空间。内存利用率严重不足。
- **标记-整理算法**：复制算法在 GC 之后存活对象较少的情况下效率比较高，但如果存活对象比较多时，会执行较多的复制操作，效率就会下降。而老年代的对象在 GC 之后的存活率就比较高，所以就有人提出了“标记-整理算法”。标记-整理算法的“标记”过程与“标记-清除算法”的标记过程一致，但标记之后不会直接清理。而是将所有存活对象都移动到内存的一端。移动结束后直接清理掉剩余部分。
- **分代回收算法**：分代收集是将内存划分成了新生代和老年代。分配的依据是对象的生存周期，或者说经历过的 GC 次数。对象创建时，一般在新生代申请内存，当经历一次 GC 之后如果对还存活，那么对象的年龄 +1。当年龄超过一定值(默认是 15，可以通过参数 -XX:MaxTenuringThreshold 来设定)后，如果对象还存活，那么该对象会进入老年代。

垃圾回收器有哪些?

垃圾收集器	类型	作用域	使用算法	特点	适用场景
Serial	串行回收	新生代	复制算法	响应速度优先	适用于单核 CPU环境下的 Client 模式
Serial Old	串行回收	老年代	标记-压缩算法	响应速度优先	适用于单核 CPU环境下的 Client 模式
ParNew	并行回收	新生代	复制算法	响应速度优先	多核 CPU环境中 Server模式下与 CMS配合使用
Parallel Scavenge	并行回收	新生代	复制算法	吞吐量优先	适用于后台运算,而交互少的场景
Parallel Old	并行回收	老年代	标记-压缩算法	吞吐量优先	适用于后台运算,而交互少的场景
CMS(Concurrent Mark-Sweep)	并发回收	老年代	标记-清除算法	响应速度优先	适用于B/S业务,也就是交互多的场景
G1(Garbage-First)	并发,并行回收(此收集器后期优化后并行方式同时存在)	新生代& 老年代(整堆收集器)	复制算法& 标记-压缩算法	响应速度优先	面向服务端的应用

- Serial 收集器（复制算法)：新生代单线程收集器，标记和清理都是单线程，优点是简单高效；
- ParNew 收集器（复制算法)：新生代收并行集器，实际上是 Serial 收集器的多线程版本，在多核 CPU 环境下有着比 Serial 更好的表现；
- Parallel Scavenge 收集器（复制算法)：新生代并行收集器，追求高吞吐量，高效利用 CPU。吞吐量 = 用户线程时间/(用户线程时间+GC 线程时间)，高吞吐量可以高效率的利用 CPU 时间，尽快完成程序的运算任务，适合后台应用等对交互相应要求不高的场景；
- Serial Old 收集器（标记-整理算法)：老年代单线程收集器，Serial 收集器的老年代版本；
- Parallel Old 收集器（标记-整理算法)： 老年代并行收集器，吞吐量优先，Parallel Scavenge 收集器的老年代版本；
- CMS(Concurrent Mark Sweep)收集器（标记-清除算法）： 老年代并行收集器，以获取最短回收停顿时间为目标的收集器，具有高并发、低停顿的特点，追求最短 GC 回收停顿时间。
- G1(Garbage First)收集器（标记-整理算法)： Java 堆并行收集器，G1 收集器是 JDK1.7 提供的一个新收集器，G1 收集器基于“标记-整理”算法实现，也就是说不会产生内存碎片。此外，G1 收集器不同于之前的收集器的一个重要特点是：G1 回收的范围是整个 Java 堆(包括新生代，老年代)，而前六种收集器回收的范围仅限于新生代或老年代

标记清除算法的缺点是什么?

主要缺点有两个：

- 一个是效率问题，标记和清除过程的效率都不高；

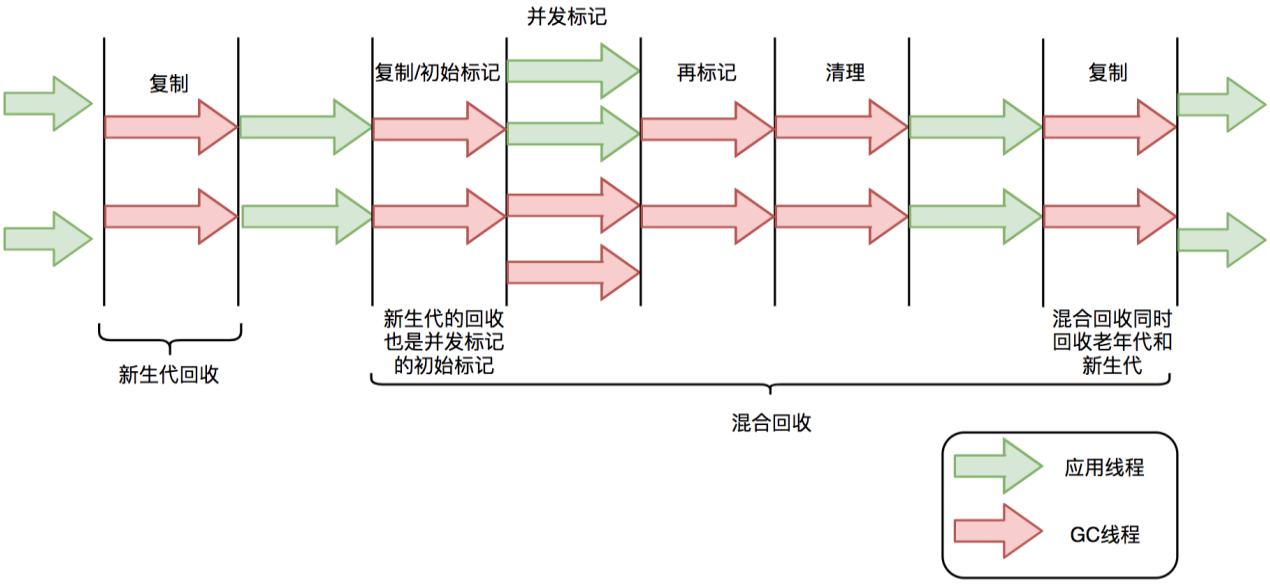
- 另外一个是空间问题，标记清除之后会产生大量不连续的内存碎片，空间碎片太多可能会导致，当程序在以后的运行过程中需要分配较大对象时无法找到足够的连续内存而不得不提前触发另一次垃圾收集动作。

垃圾回收算法哪些阶段会 stop the world?

标记-复制算法应用在 CMS 新生代（ParNew 是 CMS 默认的新生代垃圾回收器）和 G1 垃圾回收器中。标记-复制算法可以分为三个阶段：

- 标记阶段，即从 GC Roots 集合开始，标记活跃对象；
- 转移阶段，即把活跃对象复制到新的内存地址上；
- 重定位阶段，因为转移导致对象的地址发生了变化，在重定位阶段，所有指向对象旧地址的指针都要调整到对象新的地址上。

下面以 G1 为例，通过 G1 中标记-复制算法过程（G1 的 Young GC 和 Mixed GC 均采用该算法），分析 G1 停顿耗时的主要瓶颈。G1 垃圾回收周期如下图所示：



G1 的混合回收过程可以分为标记阶段、清理阶段和复制阶段。

标记阶段停顿分析

- 初始标记阶段：初始标记阶段是指从 GC Roots 出发标记全部直接子节点的过程，该阶段是 STW 的。由于 GC Roots 数量不多，通常该阶段耗时非常短。
- 并发标记阶段：并发标记阶段是指从 GC Roots 开始对堆中对象进行可达性分析，找出存活对象。该阶段是并发的，即应用线程和 GC 线程可以同时活动。并发标记耗时相对长很多，但因为不是 STW，所以我们不太关心该阶段耗时的长短。
- 再标记阶段：重新标记那些在并发标记阶段发生变化的对象。该阶段是 STW 的。

清理阶段停顿分析

- 清理阶段清点出有存活对象的分区和没有存活对象的分区，该阶段不会清理垃圾对象，也不会执行存活对象的复制。该阶段是 STW 的。

复制阶段停顿分析

- 复制算法中的转移阶段需要分配新内存和复制对象的成员变量。转移阶段是 STW 的，其中内存分配通常耗时非常短，但对象成员变量的复制耗时有可能较长，这是因为复制耗时与存活对象数量与对象复杂度成正比。对象越复杂，复制耗时越长。

四个 STW 过程中，初始标记因为只标记 GC Roots，耗时较短。再标记因为对象数少，耗时也较短。清理阶段因为内存分区数量少，耗时也较短。转移阶段要处理所有存活的对象，耗时会较长。

因此，G1 停顿时间的瓶颈主要是标记-复制中的转移阶段 STW。

minorGC、majorGC、fullGC 的区别，什么场景触发 full GC

在 Java 中，垃圾回收机制是自动管理内存的重要组成部分。根据其作用范围和触发条件的不同，可以将 GC 分为三种类型：Minor GC（也称为 Young GC）、Major GC（有时也称为 Old GC）、以及 Full GC。以下是这三种 GC 的区别和触发场景：

> Minor GC (Young GC)

- **作用范围**：只针对年轻代进行回收，包括 Eden 区和两个 Survivor 区（S0 和 S1）。
- **触发条件**：当 Eden 区空间不足时，JVM 会触发一次 Minor GC，将 Eden 区和一个 Survivor 区中的存活对象移动到另一个 Survivor 区或老年代（Old Generation）。
- **特点**：通常发生得非常频繁，因为年轻代中对象的生命周期较短，回收效率高，暂停时间相对较短。

> Major GC

- **作用范围**：主要针对老年代进行回收，但不一定只回收老年代。
- **触发条件**：当老年代空间不足时，或者系统检测到年轻代对象晋升到老年代的速度过快，可能会触发 Major GC。
- **特点**：相比 Minor GC，Major GC 发生的频率较低，但每次回收可能需要更长的时间，因为老年代中的对象存活率较高。

> Full GC

- **作用范围**：对整个堆内存（包括年轻代、老年代以及永久代/元空间）进行回收。
- **触发条件**：
    - 直接调用`System.gc()`或`Runtime.getRuntime().gc()`方法时，虽然不能保证立即执行，但 JVM 会尝试执行 Full GC。
    - Minor GC（新生代垃圾回收）时，如果存活的对象无法全部放入老年代，或者老年代空间不足以容纳存活的对象，则会触发 Full GC，对整个堆内存进行回收。
    - 当永久代（Java 8 之前的版本）或元空间（Java 8 及以后的版本）空间不足时。
- **特点**：Full GC 是最昂贵的操作，因为它需要停止所有的工作线程（Stop The World），遍历整个堆内存来查找和回收不再使用的对象，因此应尽量减少 Full GC 的触发。

垃圾回收器 CMS 和 G1 的区别?

区别一：使用的范围不一样：

- CMS 收集器是老年代的收集器，可以配合新生代的 Serial 和 ParNew 收集器一起使用
- G1 收集器收集范围是老年代和新生代。不需要结合其他收集器使用

区别二：STW 的时间：

- CMS 收集器以最小的停顿时间为目标的收集器。
- G1 收集器可预测垃圾回收 (opens new window)的停顿时间（建立可预测的停顿时间模型）

区别三： 垃圾碎片

- CMS 收集器是使用“标记-清除”算法进行的垃圾回收，容易产生内存碎片
- G1 收集器使用的是“标记-整理”算法，进行了空间整合，没有内存空间碎片。

区别四： 垃圾回收的过程不一样

CMS收集器	G1收集器
1. 初始标记	1.初始标记
2. 并发标记	2. 并发标记
3. 重新标记	3. 最终标记
4. 并发清楚	4. 筛选回收

注意这两个收集器第四阶段得不同

区别五：CMS 会产生浮动垃圾

- CMS 产生浮动垃圾过多时会退化为 serial old，效率低，因为在上图的第四阶段，CMS 清除垃圾时是并发清除的，这个时候，垃圾回收线程和用户线程同时工作会产生浮动垃圾，也就意味着 CMS 垃圾回收器必须预留一部分内存空间用于存放浮动垃圾
- 而 G1 没有浮动垃圾，G1 的筛选回收是多个垃圾回收线程并行 gc 的，没有浮动垃圾的回收，在执行‘并发清理’步骤时，用户线程也会同时产生一部分可回收对象，但是这部分可回收对象只能在下次执行清理是才会被回收。如果在清理过程中预留给用户线程的内存不足就会出现‘Concurrent Mode Failure’,一旦出现此错误时便会切换到 SerialOld 收集方式。

什么情况下使用 CMS，什么情况使用 G1?

CMS 适用场景：

- **低延迟需求**：适用于对停顿时间要求敏感的应用程序。
- **老生代收集**：主要针对老年代的垃圾回收。
- **碎片化管理**：容易出现内存碎片，可能需要定期进行 Full GC 来压缩内存空间。

G1 适用场景：

- **大堆内存**：适用于需要管理大内存堆的场景，能够有效处理数 GB 以上的堆内存。

- **对内存碎片敏感**：G1 通过紧凑整理来减少内存碎片，降低了碎片化对性能的影响。
- **比较平衡的性能**：G1 在提供较低停顿时间的同时，也保持了相对较高的吞吐量。

G1 回收器的特色是什么?

G1 的特点：

- G1 最大的特点是引入分区的思路，弱化了分代的概念。
- 合理利用垃圾收集各个周期的资源，解决了其他收集器、甚至 CMS 的众多缺陷

G1 相比较 CMS 的改进：

- **算法**： G1 基于标记--整理算法，不会产生空间碎片，在分配大对象时，不会因无法得到连续的空间，而提前触发一次 FULL GC 。
- **停顿时间可控**： G1 可以通过设置预期停顿时间（Pause Time）来控制垃圾收集时间避免应用雪崩现象。
- **并行与并发**：G1 能更充分的利用 CPU 多核环境下的硬件优势，来缩短 stop the world 的停顿时间。

GC 只会对堆进行 GC 吗?

JVM 的垃圾回收器不仅仅会对堆进行垃圾回收，它还会对方法区进行垃圾回收。

1. **堆（Heap）**： 堆是用于存储对象实例的内存区域。大部分的垃圾回收工作都发生在堆上，因为大多数对象都会被分配在堆上，而垃圾回收的重点通常也是回收堆中不再被引用的对象，以释放内存空间。
2. **方法区（Method Area）**： 方法区是用于存储类信息、常量、静态变量等数据的区域。虽然方法区中的垃圾回收与堆有所不同，但是同样存在对不再需要的常量、无用的类信息等进行清理的过程。
